

An Introduction to the Theory of Programming Languages

LAMA (post)doc days

Robin Jourde

14-15 October 2025

Outline

- 1 Introduction
 - 1.1 My motivation
 - 1.2 Programming Languages
 - 1.3 Theory?
- 2 λ -calculus
 - 2.1 Presentation
 - 2.2 Syntax
 - 2.3 Semantics
 - 2.4 Confluence
- 3 Simply Typed λ -calculus
 - 3.1 Types
 - 3.2 Typing
 - 3.3 Strong Normalization

1 Introduction

1.1 My motivation

- my PhD: “Mathematical Foundations of Skeletal Semantics”

1.1 My motivation

- my PhD: “Mathematical Foundations of Skeletal Semantics”
- semantics = giving a **meaning** to a **text**

1.1 My motivation

- my PhD: “Mathematical Foundations of Skeletal Semantics”
- semantics = giving a **meaning** to a **text**
- in computer science, text = code

1.1 My motivation

- my PhD: “Mathematical Foundations of Skeletal Semantics”
- semantics = giving a **meaning** to a **text**
- in computer science, text = code $\rightsquigarrow$ programming language

1.1 My motivation

- my PhD: “Mathematical Foundations of Skeletal Semantics”
- semantics = giving a **meaning** to a **text**
- in computer science, text = code $\rightsquigarrow$ programming language
- meaning = mathematical representation/“model”

1.1 My motivation

- my PhD: “Mathematical Foundations of Skeletal Semantics”
- semantics = giving a **meaning** to a **text**
- in computer science, text = code $\rightsquigarrow$ programming language
- meaning = mathematical representation/“model”

Remark: “Skeletal” is for next time 😬 , “Mathematical” (for my PhD) is category theory

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit
- input (and output) = “binary code”

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit
- input (and output) = “binary code”
- processor: set of instructions, encoded in binary = *assembly*

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit
- input (and output) = “binary code”
- processor: set of instructions, encoded in binary = *assembly*

Example: `ret = 0xc3, add(%rax,%rsi) = 0x4801c6, mov(%rax,%rsi) = 0x4889c6...`

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit
- input (and output) = “binary code”
- processor: set of instructions, encoded in binary = *assembly*

Example: `ret = 0xc3, add(%rax,%rsi) = 0x4801c6, mov(%rax,%rsi) = 0x4889c6...`

↪ Demo at the end if I have time...

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit
- input (and output) = “binary code”
- processor: set of instructions, encoded in binary = *assembly*

Example: `ret = 0xc3, add(%rax,%rsi) = 0x4801c6, mov(%rax,%rsi) = 0x4889c6...`

↪ Demo at the end if I have time...

- no one wants to write binary/assembly

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit
- input (and output) = “binary code”
- processor: set of instructions, encoded in binary = *assembly*

Example: `ret = 0xc3, add(%rax,%rsi) = 0x4801c6, mov(%rax,%rsi) = 0x4889c6...`

↪ Demo at the end if I have time...

- no one wants to write binary/assembly
- design **languages** to write instructions

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit
- input (and output) = “binary code”
- processor: set of instructions, encoded in binary = *assembly*

Example: `ret = 0xc3, add(%rax,%rsi) = 0x4801c6, mov(%rax,%rsi) = 0x4889c6...`

↪ Demo at the end if I have time...

- no one wants to write binary/assembly
- design **languages** to write instructions ↪ compiled to assembly

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit
- input (and output) = “binary code”
- processor: set of instructions, encoded in binary = *assembly*

Example: `ret = 0xc3, add(%rax,%rsi) = 0x4801c6, mov(%rax,%rsi) = 0x4889c6...`

↪ Demo at the end if I have time...

- no one wants to write binary/assembly
- design **languages** to write instructions ↪ compiled to assembly
 - ease: write and read ! ↪ debugging
 - correctness
 - optimization
 - complex features, different paradigms eg. functional programming, OOP, parallel computing...

1.2 Programming Languages

- computer = machine $\simeq$ big boolean circuit
- input (and output) = “binary code”
- processor: set of instructions, encoded in binary = *assembly*

Example: `ret = 0xc3, add(%rax,%rsi) = 0x4801c6, mov(%rax,%rsi) = 0x4889c6...`

↪ Demo at the end if I have time...

- no one wants to write binary/assembly
- design **languages** to write instructions ↪ compiled to assembly
 - ease: write and read ! ↪ debugging
 - correctness
 - optimization
 - complex features, different paradigms eg. functional programming, OOP, parallel computing...
 - ✕ more powerful !

1.3 Theory?

- plenty of languages: C, C++, Python, COBOL, Rust, OCaml, TeX, Gallina (Rocq), R...

1.3 Theory?

- plenty of languages: C, C++, Python, COBOL, Rust, OCaml, TeX, Gallina (Rocq), R...
- very complex, lots of features

1.3 Theory?

- plenty of languages: C, C++, Python, COBOL, Rust, OCaml, TeX, Gallina (Rocq), R...
- very complex, lots of features
- goals:

1.3 Theory?

- plenty of languages: C, C++, Python, COBOL, Rust, OCaml, TeX, Gallina (Rocq), R...
- very complex, lots of features
- goals:
 - better understand what they do
 - **verify** they do that they claim to do

1.3 Theory?

- plenty of languages: C, C++, Python, COBOL, Rust, OCaml, TeX, Gallina (Rocq), R...
- very complex, lots of features
- goals:
 - better understand what they do
 - **verify** they do that they claim to do
 - how to do it better
 - how to do new stuff

1.3 Theory?

- plenty of languages: C, C++, Python, COBOL, Rust, OCaml, TeX, Gallina (Rocq), R...
- very complex, lots of features
- goals:
 - better understand what they do
 - **verify** they do that they claim to do
 - how to do it better
 - how to do new stuff
 - find systematic methods of study

1.3 Theory?

- plenty of languages: C, C++, Python, COBOL, Rust, OCaml, TeX, Gallina (Rocq), R...
- very complex, lots of features
- goals:
 - better understand what they do
 - **verify** they do that they claim to do
 - how to do it better
 - how to do new stuff
 - find systematic methods of study
 - do some cool math 🧐

2 λ -calculus

2.1 Presentation

- a model of functional languages

2.1 Presentation

- a model of functional languages
- very simple language but... very powerful

2.1 Presentation

- a model of functional languages
- very simple language but... very powerful

Claim (Church-Turing Thesis): A mathematical function on the natural numbers can be calculated by an *effective* method if and only if it can be defined in the λ -calculus (if and only if it can be computed by a Turing machine).

2.1 Presentation

- a model of functional languages
- very simple language but... very powerful

Claim (Church-Turing Thesis): A mathematical function on the natural numbers can be calculated by an *effective* method if and only if it can be defined in the λ -calculus (if and only if it can be computed by a Turing machine).

The rest of this talk:

1. define λ -calculus
2. show some “simple” results
3. introduce a typed version of λ -calculus
4. present the strong normalization of this new calculus and why it’s nice

2.2 Syntax

Definition (λ -terms):

$$t, u \dots ::= x \mid t \ u \mid \lambda x. t$$

$x \in X$ a set of variables. We consider terms equal up to renaming of **bound** variables.

2.2 Syntax

Definition (λ -terms):

$$t, u \dots ::= x \mid t \ u \mid \lambda x. t$$

$x \in X$ a set of variables. We consider terms equal up to renaming of **bound** variables.

Example:

$$\text{id} = \lambda x. x = \lambda y. y$$

2.2 Syntax

Definition (λ -terms):

$$t, u \dots ::= x \mid t \ u \mid \lambda x. t$$

$x \in X$ a set of variables. We consider terms equal up to renaming of **bound** variables.

Example:

$$\text{id} = \lambda x. x = \lambda y. y \quad \lambda y. x = \lambda z. x \neq \lambda y. z$$

2.2 Syntax

Definition (λ -terms):

$$t, u \dots ::= x \mid t \ u \mid \lambda x. t$$

$x \in X$ a set of variables. We consider terms equal up to renaming of **bound** variables.

Example:

$$\text{id} = \lambda x. x = \lambda y. y \quad \lambda y. x = \lambda z. x \neq \lambda y. z \quad \lambda x. y \ (z \ x) \quad \lambda x. (y \ z) \ x = \lambda x. y \ z \ x$$

2.2 Syntax

Definition (λ -terms):

$$t, u \dots ::= x \mid t \ u \mid \lambda x. t$$

$x \in X$ a set of variables. We consider terms equal up to renaming of **bound** variables.

Example:

$$\begin{aligned} \text{id} &= \lambda x. x = \lambda y. y & \lambda y. x &= \lambda z. x \neq \lambda y. z & \lambda x. y \ (z \ x) & \lambda x. (y \ z) \ x = \lambda x. y \ z \ x \\ & & & & & (\lambda x. x \ \text{id}) \ (y \ (\lambda x. x \ y)) \end{aligned}$$

2.2 Syntax

Definition (λ -terms):

$$t, u \dots ::= x \mid t \ u \mid \lambda x. t$$

$x \in X$ a set of variables. We consider terms equal up to renaming of **bound** variables.

Example:

$$\text{id} = \lambda x. x = \lambda y. y \quad \lambda y. x = \lambda z. x \neq \lambda y. z \quad \lambda x. y \ (z \ x) \quad \lambda x. (y \ z) \ x = \lambda x. y \ z \ x$$

$$(\lambda x. x \ \text{id}) \ (y \ (\lambda x. x \ y)) \quad \omega = \lambda x. x \ x \quad \Omega = \omega \ \omega = (\lambda x. x \ x) \ (\lambda x. x \ x)$$

2.3 Semantics

Reduction relation $t \rightarrow_{\beta} u$: “if I have the term t and I do one step of computation I end up with u ”

2.3 Semantics

Reduction relation $t \rightarrow_{\beta} u$: “if I have the term t and I do one step of computation I end up with u ”

Definition (β -reduction): Inductively,

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta$$

where $t[x/u]$ is the term t where each occurrence of x is replaced by u

2.3 Semantics

Reduction relation $t \rightarrow_{\beta} u$: “if I have the term t and I do one step of computation I end up with u ”

Definition (β -reduction): Inductively,

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta \quad \frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \lambda \quad \frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \text{appl} \quad \frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \text{appr}$$

where $t[x/u]$ is the term t where each occurrence of x is replaced by u

2.3 Semantics

Reduction relation $t \rightarrow_{\beta} u$: “if I have the term t and I do one step of computation I end up with u ”

Definition (β -reduction): Inductively,

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta \quad \frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \lambda \quad \frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \text{appl} \quad \frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \text{appr}$$

where $t[x/u]$ is the term t where each occurrence of x is replaced by u

Definition (Normal Form): t is a **normal form** if there is no u such that $t \rightarrow_{\beta} u$.
We write $t \nrightarrow_{\beta}$.

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta$$

$$\frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \lambda$$

$$\frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \text{appl}$$

$$\frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \text{appr}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta}$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta$$

$$\frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \lambda$$

$$\frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \text{appl}$$

$$\frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \text{appr}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta$$

$$\frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \lambda$$

$$\frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \text{appl}$$

$$\frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \text{appr}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

$$f = \lambda x.\lambda y.y x \quad f t \rightarrow_{\beta} \lambda y.y t$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta$$

$$\frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \lambda$$

$$\frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \text{appl}$$

$$\frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \text{appr}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

$$f = \lambda x.\lambda y.y x \quad f t \rightarrow_{\beta} \lambda y.y t$$

$$(\lambda z.f z) \omega \rightarrow_{\beta} f$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta$$

$$\frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \lambda$$

$$\frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \text{appl}$$

$$\frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \text{appr}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

$$f = \lambda x.\lambda y.y x \quad f t \rightarrow_{\beta} \lambda y.y t$$

$$(\lambda z.f z) \omega \rightarrow_{\beta} f \omega \rightarrow_{\beta} \lambda y.y \omega$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta$$

$$\frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \lambda$$

$$\frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \text{appl}$$

$$\frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \text{appr}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

$$f = \lambda x.\lambda y.y x \quad f t \rightarrow_{\beta} \lambda y.y t$$

$$(\lambda z.f z) \omega \rightarrow_{\beta} f \omega \rightarrow_{\beta} \lambda y.y \omega \quad (\lambda z.f z) \omega \rightarrow_{\beta} (\lambda z.\lambda y.y z) \omega$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]} \beta$$

$$\frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \lambda$$

$$\frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \text{appl}$$

$$\frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \text{appr}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

$$f = \lambda x.\lambda y.y x \quad f t \rightarrow_{\beta} \lambda y.y t$$

$$(\lambda z.f z) \omega \rightarrow_{\beta} f \omega \rightarrow_{\beta} \lambda y.y \omega \quad (\lambda z.f z) \omega \rightarrow_{\beta} (\lambda z.\lambda y.y z) \omega \rightarrow_{\beta} \lambda y.y \omega$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]}_{\beta} \quad \frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'}_{\lambda} \quad \frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u}_{\text{appl}} \quad \frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'}_{\text{appr}}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

$$f = \lambda x.\lambda y.y x \quad f t \rightarrow_{\beta} \lambda y.y t$$

$$(\lambda z.f z) \omega \rightarrow_{\beta} f \omega \rightarrow_{\beta} \lambda y.y \omega \quad (\lambda z.f z) \omega \rightarrow_{\beta} (\lambda z.\lambda y.y z) \omega \rightarrow_{\beta} \lambda y.y \omega$$

$$x \nrightarrow_{\beta} \quad \text{id} \nrightarrow_{\beta} \quad \lambda x.t \nrightarrow_{\beta}$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]}_{\beta} \quad \frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'}_{\lambda} \quad \frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u}_{\text{appl}} \quad \frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'}_{\text{appr}}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

$$f = \lambda x.\lambda y.y x \quad f t \rightarrow_{\beta} \lambda y.y t$$

$$(\lambda z.f z) \omega \rightarrow_{\beta} f \omega \rightarrow_{\beta} \lambda y.y \omega \quad (\lambda z.f z) \omega \rightarrow_{\beta} (\lambda z.\lambda y.y z) \omega \rightarrow_{\beta} \lambda y.y \omega$$

$$x \nrightarrow_{\beta} \quad \text{id} \nrightarrow_{\beta} \quad \lambda x.t \nrightarrow_{\beta}$$

$$\Omega = \omega \omega = (\lambda x.x x) \omega \rightarrow_{\beta}$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]}_{\beta} \quad \frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'}_{\lambda} \quad \frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u}_{\text{appl}} \quad \frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'}_{\text{appr}}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

$$f = \lambda x.\lambda y.y x \quad f t \rightarrow_{\beta} \lambda y.y t$$

$$(\lambda z.f z) \omega \rightarrow_{\beta} f \omega \rightarrow_{\beta} \lambda y.y \omega \quad (\lambda z.f z) \omega \rightarrow_{\beta} (\lambda z.\lambda y.y z) \omega \rightarrow_{\beta} \lambda y.y \omega$$

$$x \nrightarrow_{\beta} \quad \text{id} \nrightarrow_{\beta} \quad \lambda x.t \nrightarrow_{\beta}$$

$$\Omega = \omega \omega = (\lambda x.x x) \omega \rightarrow_{\beta} \omega \omega$$

2.3 Semantics

$$\frac{}{(\lambda x.t) u \rightarrow_{\beta} t[x/u]}_{\beta} \quad \frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'}_{\lambda} \quad \frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u}_{\text{appl}} \quad \frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'}_{\text{appr}}$$

Example:

$$\text{id } y = (\lambda x.x) y \rightarrow_{\beta} x[x/y] = y$$

$$f = \lambda x.\lambda y.y x \quad f t \rightarrow_{\beta} \lambda y.y t$$

$$(\lambda z.f z) \omega \rightarrow_{\beta} f \omega \rightarrow_{\beta} \lambda y.y \omega \quad (\lambda z.f z) \omega \rightarrow_{\beta} (\lambda z.\lambda y.y z) \omega \rightarrow_{\beta} \lambda y.y \omega$$

$$x \nrightarrow_{\beta} \quad \text{id} \nrightarrow_{\beta} \quad \lambda x.t \nrightarrow_{\beta}$$

$$\Omega = \omega \omega = (\lambda x.x x) \omega \rightarrow_{\beta} \omega \omega \rightarrow_{\beta} \omega \omega \rightarrow_{\beta} \dots$$

2.4 Confluence

Definition: $t \rightarrow_{\beta}^* u \Leftrightarrow t \rightarrow_{\beta} \dots \rightarrow_{\beta} u$

Theorem (Confluence): For any terms t, u, u' such that $t \rightarrow_{\beta} u$ and $t \rightarrow_{\beta} u'$, there exists a term v such that $u \rightarrow_{\beta}^* v$ and $u' \rightarrow_{\beta}^* v$.

2.4 Confluence

Definition: $t \rightarrow_{\beta}^* u \Leftrightarrow t \rightarrow_{\beta} \dots \rightarrow_{\beta} u$

Theorem (Confluence): For any terms t, u, u' such that $t \rightarrow_{\beta} u$ and $t \rightarrow_{\beta} u'$, there exists a term v such that $u \rightarrow_{\beta}^* v$ and $u' \rightarrow_{\beta}^* v$.

Corollary: If $t \rightarrow_{\beta}^* u \nrightarrow_{\beta}$ then such a u is unique.

2.4 Confluence

Definition: $t \rightarrow_{\beta}^* u \Leftrightarrow t \rightarrow_{\beta} \dots \rightarrow_{\beta} u$

Theorem (Confluence): For any terms t, u, u' such that $t \rightarrow_{\beta} u$ and $t \rightarrow_{\beta} u'$, there exists a term v such that $u \rightarrow_{\beta}^* v$ and $u' \rightarrow_{\beta}^* v$.

Corollary: If $t \rightarrow_{\beta}^* u \nrightarrow_{\beta}$ then such a u is unique.

Proof of Confluence: Introduce a new relation $t \twoheadrightarrow_{\beta} u$, show it has the “diamond property” and show it entails confluence for $\rightarrow_{\beta}$. See rest of this section. $\square$

2.4 Confluence

Definition (Simultaneous reduction):

$$\frac{t \twoheadrightarrow_{\beta} t' \quad u \twoheadrightarrow_{\beta} u'}{(\lambda x.t) u \twoheadrightarrow_{\beta} t'[x/u']} \beta \quad \frac{t \twoheadrightarrow_{\beta} t'}{\lambda x.t \twoheadrightarrow_{\beta} \lambda x.t'} \lambda \quad \frac{t \twoheadrightarrow_{\beta} t' \quad u \twoheadrightarrow_{\beta} u'}{t u \twoheadrightarrow_{\beta} t' u'} \text{app} \quad \frac{}{t \twoheadrightarrow_{\beta} t} \text{id}$$

2.4 Confluence

Definition (Simultaneous reduction):

$$\frac{t \twoheadrightarrow_{\beta} t' \quad u \twoheadrightarrow_{\beta} u'}{(\lambda x.t) u \twoheadrightarrow_{\beta} t'[x/u']} \beta \quad \frac{t \twoheadrightarrow_{\beta} t'}{\lambda x.t \twoheadrightarrow_{\beta} \lambda x.t'} \lambda \quad \frac{t \twoheadrightarrow_{\beta} t' \quad u \twoheadrightarrow_{\beta} u'}{t u \twoheadrightarrow_{\beta} t' u'} \text{app} \quad \frac{}{t \twoheadrightarrow_{\beta} t} \text{id}$$

Lemma: $t \rightarrow_{\beta} t' \implies t \twoheadrightarrow_{\beta} t' \implies t \rightarrow_{\beta}^* t'.$

2.4 Confluence

Definition (Simultaneous reduction):

$$\frac{t \twoheadrightarrow_{\beta} t' \quad u \twoheadrightarrow_{\beta} u'}{(\lambda x.t) u \twoheadrightarrow_{\beta} t'[x/u']} \beta \quad \frac{t \twoheadrightarrow_{\beta} t'}{\lambda x.t \twoheadrightarrow_{\beta} \lambda x.t'} \lambda \quad \frac{t \twoheadrightarrow_{\beta} t' \quad u \twoheadrightarrow_{\beta} u'}{t u \twoheadrightarrow_{\beta} t' u'} \text{app} \quad \frac{}{t \twoheadrightarrow_{\beta} t} \text{id}$$

Lemma: $t \rightarrow_{\beta} t' \implies t \twoheadrightarrow_{\beta} t' \implies t \rightarrow_{\beta}^* t'$.

Lemma (Diamond property): For any terms t, u, u' such that $t \twoheadrightarrow_{\beta} u$ and $t \twoheadrightarrow_{\beta} u'$, there exists a term v such that $u \twoheadrightarrow_{\beta} v$ and $u' \twoheadrightarrow_{\beta} v$.

2.4 Confluence

Proof of confluence:

$$\begin{aligned} t \rightarrow_{\beta} u \wedge t \rightarrow_{\beta} u' &\implies t \twoheadrightarrow_{\beta} u \wedge t \twoheadrightarrow_{\beta} u' \\ &\implies \exists v, u \twoheadrightarrow_{\beta} v \wedge u' \twoheadrightarrow_{\beta} v \\ &\implies \exists v, u \rightarrow_{\beta}^* v \wedge u' \rightarrow_{\beta}^* v \end{aligned}$$



2.4 Confluence

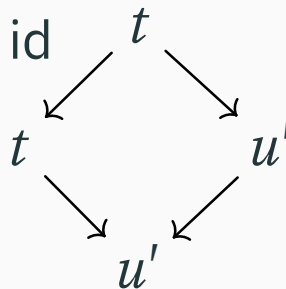
Proof of confluence:

$$\begin{aligned} t \rightarrow_{\beta} u \wedge t \rightarrow_{\beta} u' &\implies t \twoheadrightarrow_{\beta} u \wedge t \twoheadrightarrow_{\beta} u' \\ &\implies \exists v, u \twoheadrightarrow_{\beta} v \wedge u' \twoheadrightarrow_{\beta} v \\ &\implies \exists v, u \rightarrow_{\beta}^* v \wedge u' \rightarrow_{\beta}^* v \end{aligned}$$

□

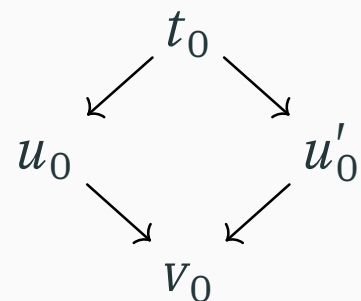
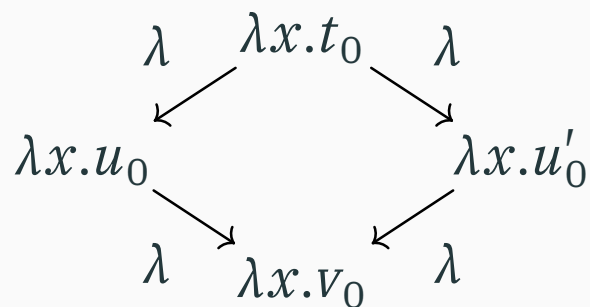
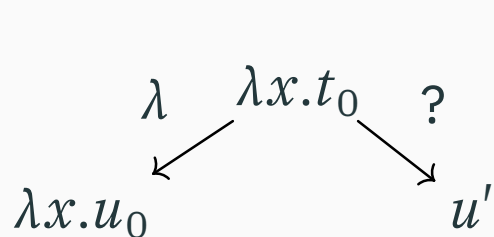
Proof of the Diamond Property lemma: By induction on $t \twoheadrightarrow_{\beta} u$.

id vs. anything $u = t$



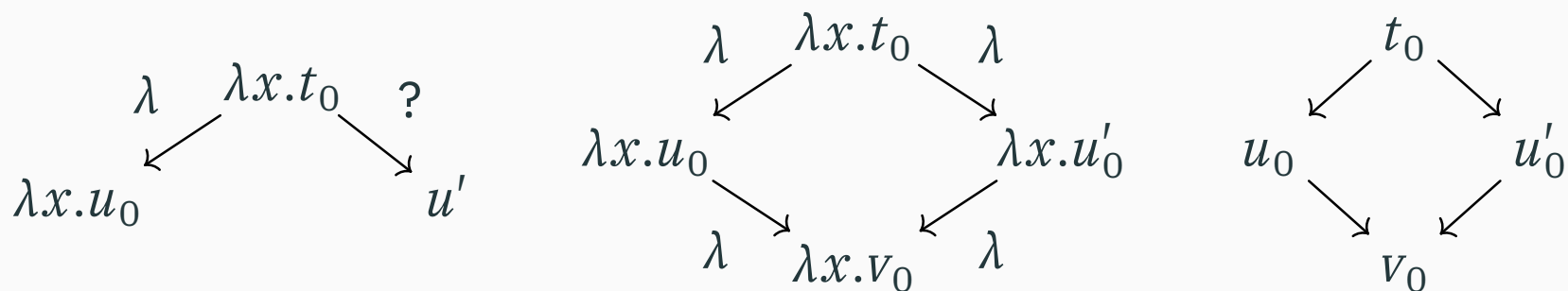
2.4 Confluence

λ vs. anything $t = \lambda x.t_0$, $u = \lambda x.u_0$ and $t_0 \rightarrow_{\beta} u_0$

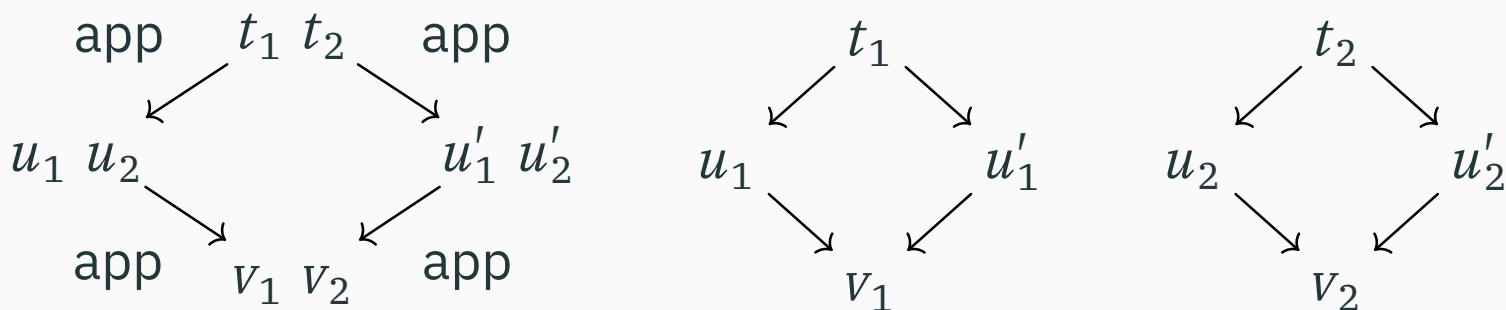


2.4 Confluence

λ vs. anything $t = \lambda x.t_0$, $u = \lambda x.u_0$ and $t_0 \rightarrow_{\beta} u_0$

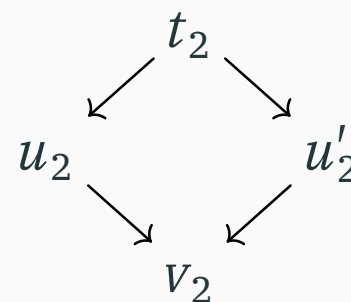
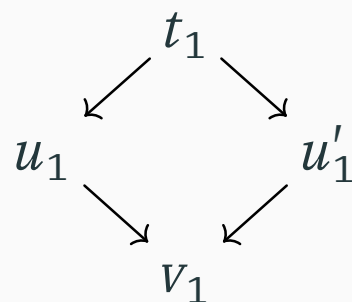
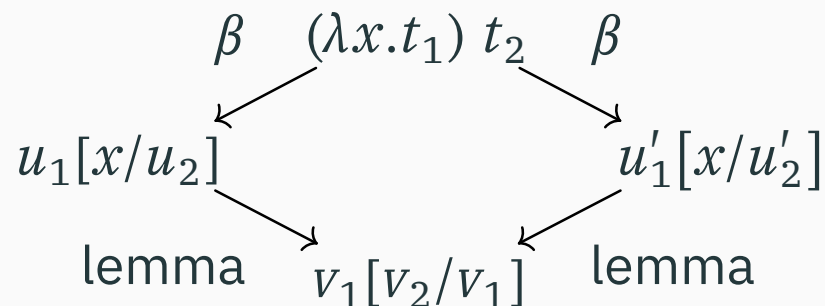


app vs. app $t = t_1 t_2$, $u = u_1 u_2$, $u' = u'_1 u'_2$ and ...



2.4 Confluence

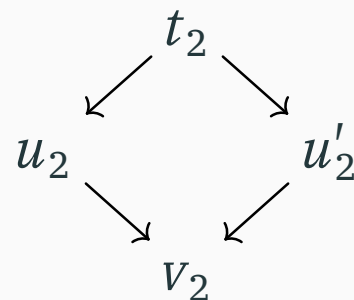
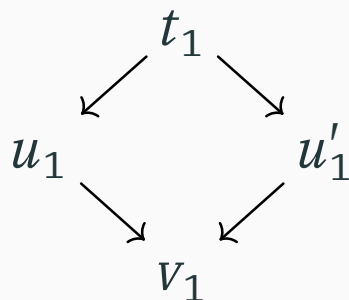
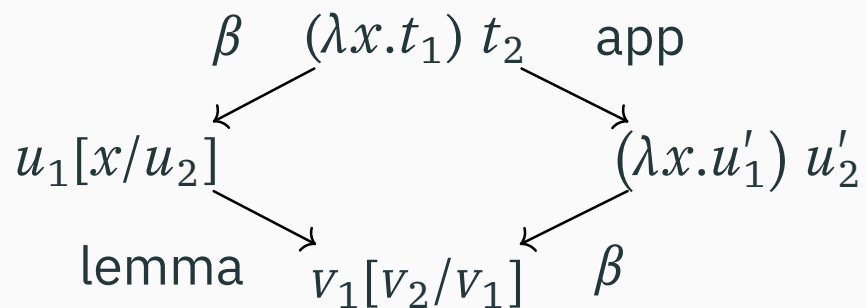
β vs. β $t = (\lambda x.t_1) t_2$ and ...



Lemma: $t \twoheadrightarrow_{\beta} t' \wedge u \twoheadrightarrow_{\beta} u' \Rightarrow t[x/u] \twoheadrightarrow_{\beta} t'[x/u']$.

2.4 Confluence

β vs. app $t = (\lambda x.t_1) t_2$ and ...



3 Simply Typed λ -calculus

3.1 Types

Idea:

- for a function, what kind of input/output is expected

3.1 Types

Idea:

- for a function, what kind of input/output is expected
- “static analysis” (without running the program)

3.1 Types

Idea:

- for a function, what kind of input/output is expected
- “static analysis” (without running the program)
- pros: help the programmer + forbids some errors

3.1 Types

Idea:

- for a function, what kind of input/output is expected
- “static analysis” (without running the program)
- pros: help the programmer + forbids some errors

Definition (Types): For some set $\mathcal{T}$ of base types,

$$A, B \dots ::= T \in \mathcal{T} \mid A \rightarrow B$$

3.1 Types

Idea:

- for a function, what kind of input/output is expected
- “static analysis” (without running the program)
- pros: help the programmer + forbids some errors

Definition (Types): For some set $\mathcal{T}$ of base types,

$$A, B... ::= T \in \mathcal{T} \mid A \rightarrow B$$

Example: with $\mathcal{T} = \{\text{nat}, \text{bool}, \text{unit}\}$

unit bool

3.1 Types

Idea:

- for a function, what kind of input/output is expected
- “static analysis” (without running the program)
- pros: help the programmer + forbids some errors

Definition (Types): For some set $\mathcal{T}$ of base types,

$$A, B \dots ::= T \in \mathcal{T} \mid A \rightarrow B$$

Example: with $\mathcal{T} = \{\text{nat}, \text{bool}, \text{unit}\}$

unit bool nat $\rightarrow$ nat bool $\rightarrow$ unit

3.1 Types

Idea:

- for a function, what kind of input/output is expected
- “static analysis” (without running the program)
- pros: help the programmer + forbids some errors

Definition (Types): For some set $\mathcal{T}$ of base types,

$$A, B \dots ::= T \in \mathcal{T} \mid A \rightarrow B$$

Example: with $\mathcal{T} = \{\text{nat}, \text{bool}, \text{unit}\}$

unit bool nat $\rightarrow$ nat bool $\rightarrow$ unit nat $\rightarrow$ (nat $\rightarrow$ nat)

(nat $\rightarrow$ bool) $\rightarrow$ (bool $\rightarrow$ nat) unit $\rightarrow$ ((bool $\rightarrow$ unit) $\rightarrow$ nat)

3.2 Typing

Definition (Context): List of “typed variables” $(x : A) :$

$$\Gamma ::= (x : A), (y : B), (z : C) \dots$$

Definition (Typing judgment): “Term t has type A in context Γ ”

$$\Gamma \vdash t : A$$

3.2 Typing

Definition (Context): List of “typed variables” $(x : A) :$

$$\Gamma ::= (x : A), (y : B), (z : C) \dots$$

Definition (Typing judgment): “Term t has type A in context Γ ”

$$\Gamma \vdash t : A$$

Definition (Typing rules):

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A} \text{var}$$

$$\frac{\Gamma \vdash t : A \rightarrow B \quad \Gamma \vdash u : A}{\Gamma \vdash t u : B} \text{app}$$

$$\frac{\Gamma, (x : A) \vdash t : B}{\Gamma \vdash \lambda x. t : A \rightarrow B} \lambda$$

3.2 Typing

Example: $\emptyset \vdash \lambda x.x : ?$

$\emptyset \vdash \lambda x.\lambda y.y\ x : ?$

$k : ?, z : ? \vdash (\lambda x.\lambda y.z\ x)\ k : ?$

3.2 Typing

Example: $\emptyset \vdash \lambda x.x : ?$

$\emptyset \vdash \lambda x.\lambda y.y\ x : ?$

$k : ?, z : ? \vdash (\lambda x.\lambda y.z\ x)\ k : ?$

$$\frac{(x : A) \in (x : A)}{(x : A) \vdash x : A} \text{var}$$
$$\frac{(x : A) \vdash x : A}{\emptyset \vdash \lambda x.x : A \rightarrow A} \lambda$$

3.2 Typing

Example: $\emptyset \vdash \lambda x.x : ?$

$$\frac{(x : A) \in (x : A)}{(x : A) \vdash x : A} \text{var}$$
$$\frac{(x : A) \vdash x : A}{\emptyset \vdash \lambda x.x : A \rightarrow A} \lambda$$

$\emptyset \vdash \lambda x.\lambda y.y x : ?$

$$\frac{(x : A), (y : A \rightarrow B) \vdash y x : B}{(x : A) \vdash \lambda y.y x : (A \rightarrow B) \rightarrow B} \lambda$$
$$\frac{(x : A) \vdash \lambda y.y x : (A \rightarrow B) \rightarrow B}{\emptyset \vdash \lambda x.\lambda y.y x : A \rightarrow (A \rightarrow B) \rightarrow B} \lambda$$

$k : ?, z : ? \vdash (\lambda x.\lambda y.z x) k : ?$

$$\frac{(x : A), (y : A \rightarrow B) \vdash y : A \rightarrow B \quad (x : A), (y : A \rightarrow B) \vdash x : A}{(x : A), (y : A \rightarrow B) \vdash y x : B} \text{app}$$

3.2 Typing

Example: $\emptyset \vdash \lambda x.x : ?$

$$\emptyset \vdash \lambda x. \lambda y. y \ x : ?$$
$$k : ?, z : ? \vdash (\lambda x. \lambda y. z \ x) \ k : ?$$
$$\frac{\frac{(x : A) \in (x : A)}{(x : A) \vdash x : A} \text{var}}{\emptyset \vdash \lambda x. x : A \rightarrow A} \lambda$$
$$\frac{\frac{\frac{(x : A), (y : A \rightarrow B) \vdash y : A \rightarrow B \quad (x : A), (y : A \rightarrow B) \vdash x : A}{\text{app}}}{(x : A), (y : A \rightarrow B) \vdash y \ x : B}{\lambda} \quad \frac{(x : A) \vdash \lambda y. y \ x : (A \rightarrow B) \rightarrow B}{\lambda}$$
$$\begin{array}{c}
\text{var} \quad \frac{}{..., z : A \rightarrow B \vdash z : A \rightarrow B} \quad \text{var} \quad \frac{}{..., x : A \vdash x : A} \\
\text{app} \quad \frac{}{} \\
\lambda \quad \frac{k : A, z : A \rightarrow B, x : A, y : C \vdash z x : B}{k : A, z : A \rightarrow B, x : A \vdash \lambda y. z x : C \rightarrow B} \\
\lambda \quad \frac{k : A, z : A \rightarrow B, x : A \vdash \lambda y. z x : C \rightarrow B}{k : A, z : A \rightarrow B \vdash \lambda x. \lambda y. z x : A \rightarrow (C \rightarrow B)} \quad \text{var} \quad \frac{}{k : A, z : A \rightarrow B \vdash k : A} \\
\text{app} \quad \frac{k : A, z : A \rightarrow B \vdash \lambda x. \lambda y. z x : A \rightarrow (C \rightarrow B) \quad k : A, z : A \rightarrow B \vdash k : A}{k : A, z : A \rightarrow B \vdash (\lambda x. \lambda y. z x) k : C \rightarrow B}
\end{array}$$

3.2 Typing

Remark: Recall $\Omega = \omega \ \omega$ with $\omega = \lambda x.x \ x$ such that $\Omega \rightarrow_{\beta} \Omega \rightarrow_{\beta} \dots$. What is the type of ω ?

3.2 Typing

Remark: Recall $\Omega = \omega \ \omega$ with $\omega = \lambda x.x \ x$ such that $\Omega \rightarrow_{\beta} \Omega \rightarrow_{\beta} \dots$. What is the type of ω ?

$$\frac{\frac{\Gamma, (x : A) \vdash x : C \rightarrow B \quad \Gamma, (x : A) \vdash x : C}{\Gamma, (x : A) \vdash x \ x : B} \text{app}}{\Gamma \vdash \lambda x.x \ x : A \rightarrow B} \lambda$$

3.2 Typing

Remark: Recall $\Omega = \omega \ \omega$ with $\omega = \lambda x.x \ x$ such that $\Omega \rightarrow_{\beta} \Omega \rightarrow_{\beta} \dots$. What is the type of ω ?

$$\frac{\Gamma, (x : A) \vdash x : C \rightarrow B \quad \Gamma, (x : A) \vdash x : C}{\Gamma, (x : A) \vdash x \ x : B} \text{app}$$
$$\frac{\Gamma, (x : A) \vdash x \ x : B}{\Gamma \vdash \lambda x.x \ x : A \rightarrow B} \lambda$$

$C \rightarrow B = A = C \rightsquigarrow$ impossible, ω cannot be typed ! Therefore, $\Omega = \omega \ \omega$ can't be typed either.

3.3 Strong Normalization

Theorem: Simply typed λ -calculus is strongly normalizing:

$$\forall t, (\exists \Gamma, \exists A, \Gamma \vdash t : A) \implies (\exists u, t \rightarrow_{\beta}^* u \nrightarrow_{\beta})$$

3.3 Strong Normalization

Theorem: Simply typed λ -calculus is strongly normalizing:

$$\forall t, (\exists \Gamma, \exists A, \Gamma \vdash t : A) \implies (\exists u, t \rightarrow_{\beta}^* u \nrightarrow_{\beta})$$

Consequences:

- typed λ -calculus is more robust than untyped λ -calculus 😊

3.3 Strong Normalization

Theorem: Simply typed λ -calculus is strongly normalizing:

$$\forall t, (\exists \Gamma, \exists A, \Gamma \vdash t : A) \implies (\exists u, t \rightarrow_{\beta}^* u \nrightarrow_{\beta})$$

Consequences:

- typed λ -calculus is more robust than untyped λ -calculus 😊
- but no longer Turing complete 😞

3.3 Strong Normalization

Theorem: Simply typed λ -calculus is strongly normalizing:

$$\forall t, (\exists \Gamma, \exists A, \Gamma \vdash t : A) \implies (\exists u, t \rightarrow_{\beta}^* u \nrightarrow_{\beta})$$

Consequences:

- typed λ -calculus is more robust than untyped λ -calculus 😊
- but no longer Turing complete 😞